

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 7 Computer Peripherals and I/O

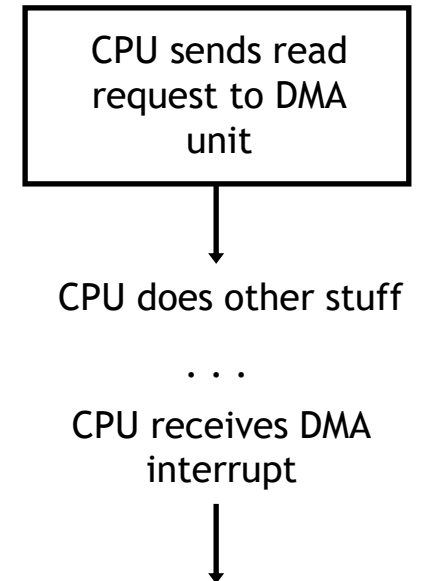
Disclaimer



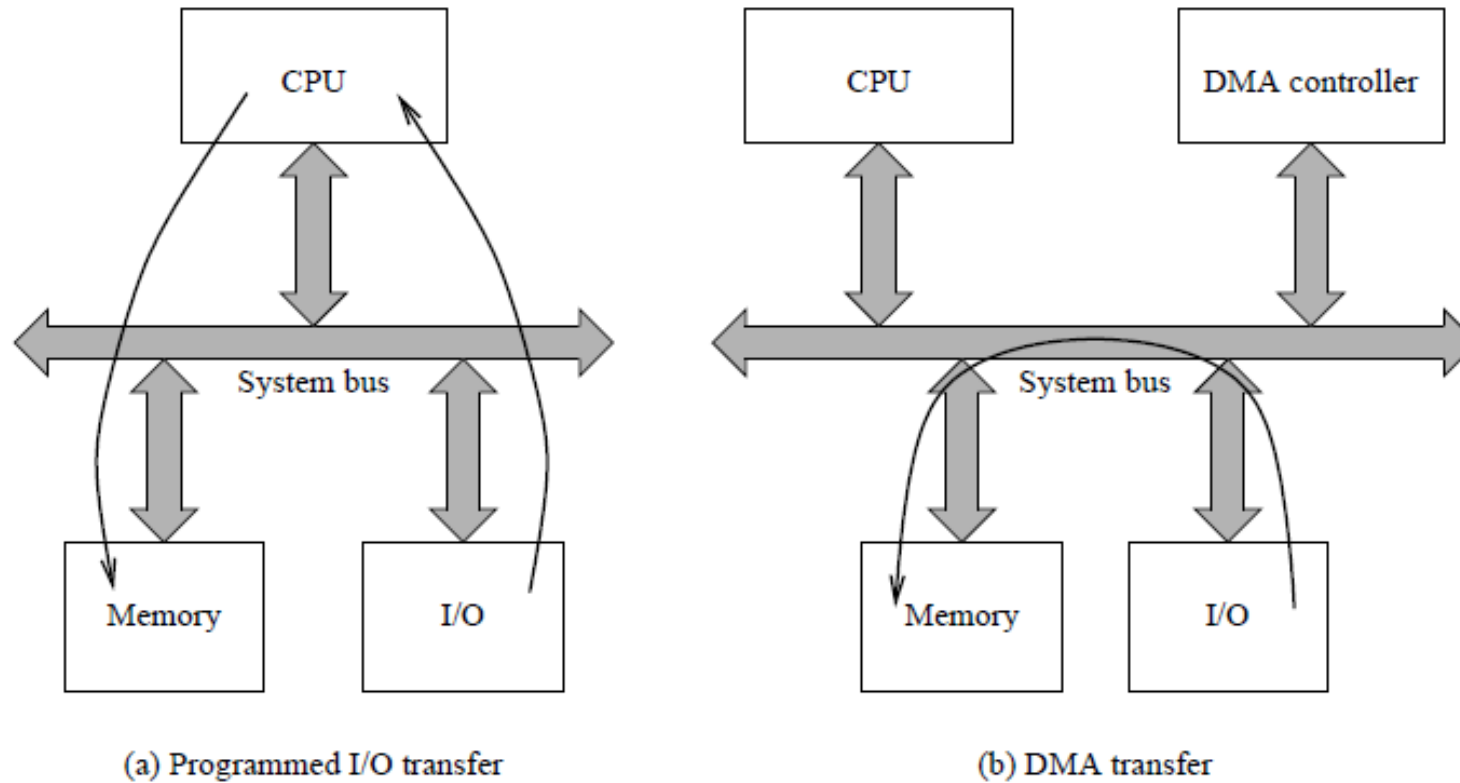
- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Direct Memory Access, DMA

- One final method of data transfer is to introduce a **direct memory access**, or **DMA**, controller
- The DMA controller is a simple processor which does most of the functions that the CPU would otherwise have to handle
 - The CPU asks the DMA controller to transfer data between a device and main memory. After that, the CPU can continue with other tasks
 - The DMA controller issues requests to the right I/O device, waits, and manages the transfers between the device and main memory
 - Once finished, the DMA controller interrupts the CPU
- DMA relieves the CPU of the low-level data transfer chore. But it needs hardware support as it is implemented by using a DMA controller.

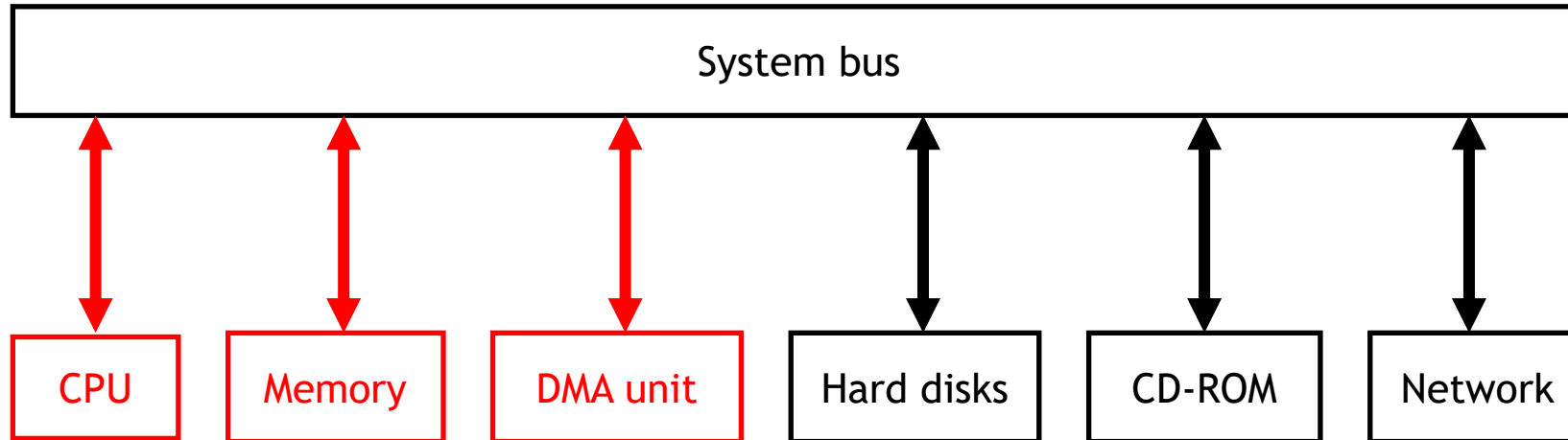


Programmed I/O vs DMA



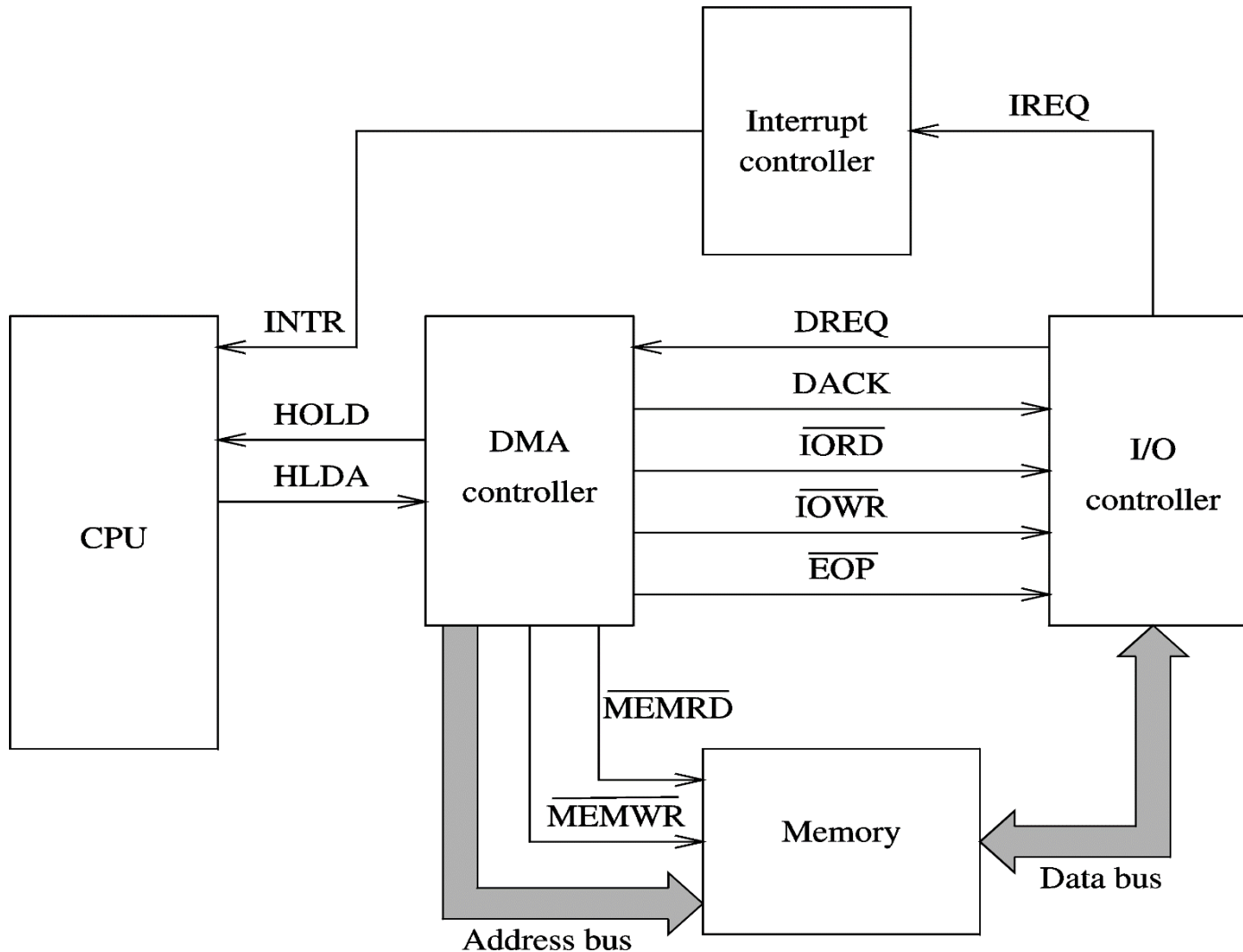
- (a) In Programmed I/O, data are read by the processor and then written to the memory.
- (b) In DMA transfer, the DMA controller generates the control signals to transfer data directly between the I/O device and memory.

Direct Memory Access



Since both the processor and the DMA controller may need to access main memory, some form of **arbitration** is required

Direct Memory Access



DREQ(DMA Request) - Used by a device to request a DMA transfer

HOLD - DMA controller request the CPU to release the bus

HLDA(Hold Acknowledge) - CPU acknowledges the HOLD request and releases control of the system bus

DACK(DMA Acknowledge) - Used by DMA controller to acknowledge DMA request and initiate DMA transfer

EOP - End of Process

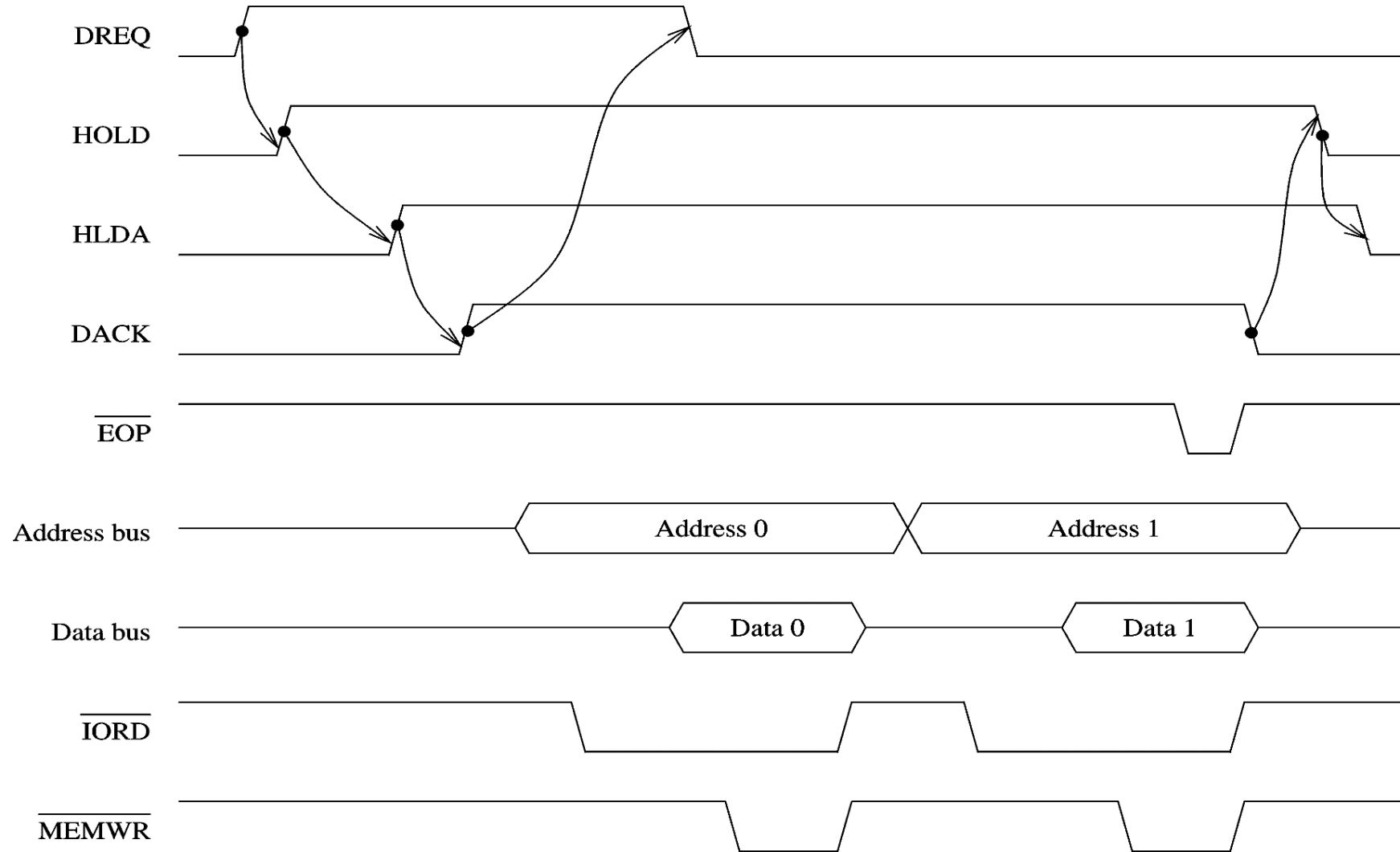
IORD - I/O Read

IOWR - I/O Write

MEMRD - Memory Read

MEMWR - Memory Write

Direct Memory Access



Bus Arbitration

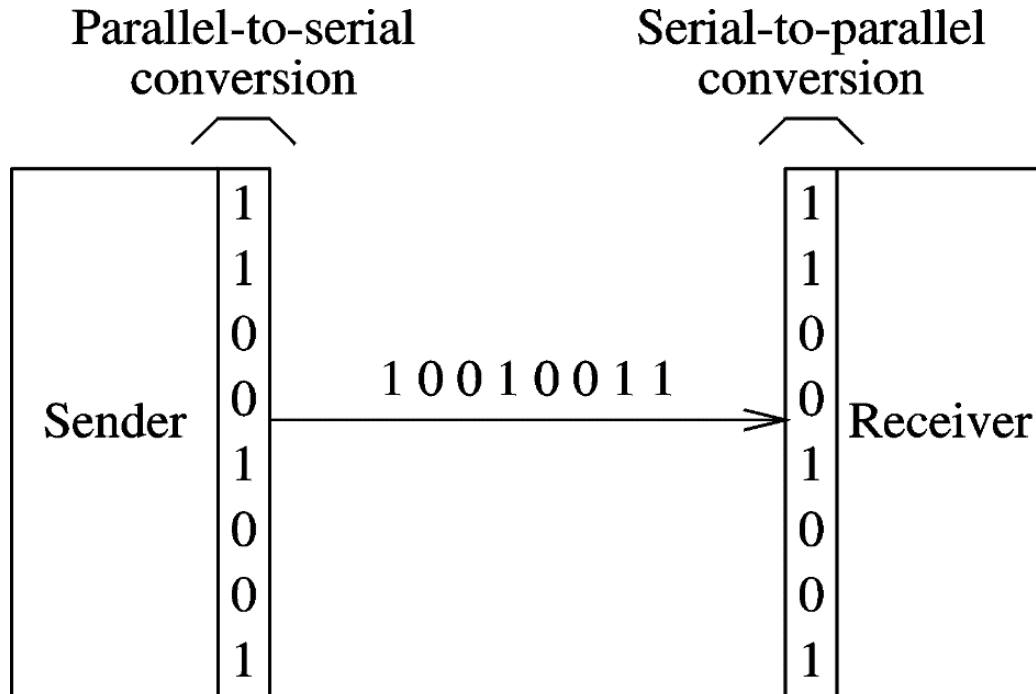
- More than one bus master can request the bus
 - Need an arbitration mechanism to allocate the bus
- Bus arbitration can be done either
 - Statically
 - Dynamically
- Static arbitration
 - Done in a **predetermined way**
 - Easy to implement
 - Does not take needs into account
 - Poor utilization - Bus could be assigned even when not needed
- Dynamic bus arbitration
 - Bus allocated only **in response to a request**
 - Each master is equipped with
 - Bus request line
 - Bus grant line
 - A master uses the bus request line to let others know that it needs the bus
 - Before a master can use the bus, it must receive permission to use the bus via the bus grant line

** A master device will initiate the transaction, and a slave device will respond to the master's request

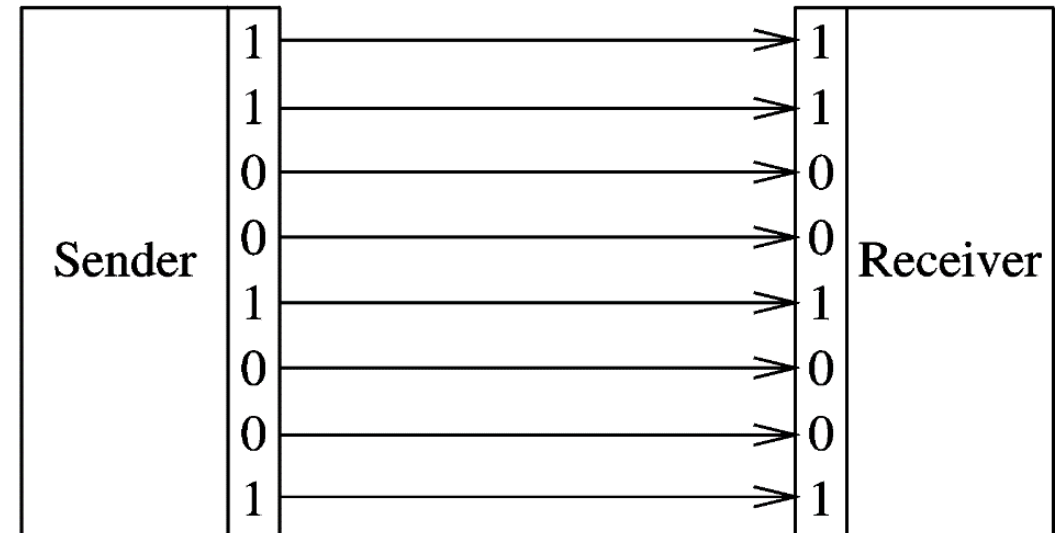
Data Transmission

- Transfer of data may be parallel or serial.
- **Parallel data transfer:**
 - Each bit has its own path, and the entire message is **transmitted at one time**.
 - **Faster**, because multiple signal lines operate in parallel.
 - Used for **short distances** and when speed is important.
 - **Data skew problem** – timing imbalance or misalignment between transmitted data on different channels
- **Serial data transfer:**
 - Each bit is sent in sequence, **one at a time**.
 - Slower, but **less expensive** since it requires only one conductor.
 - Become increasingly important: Ease of connecting smaller cables, and **less signal skew** problem as data rates increase.

Data Transmission

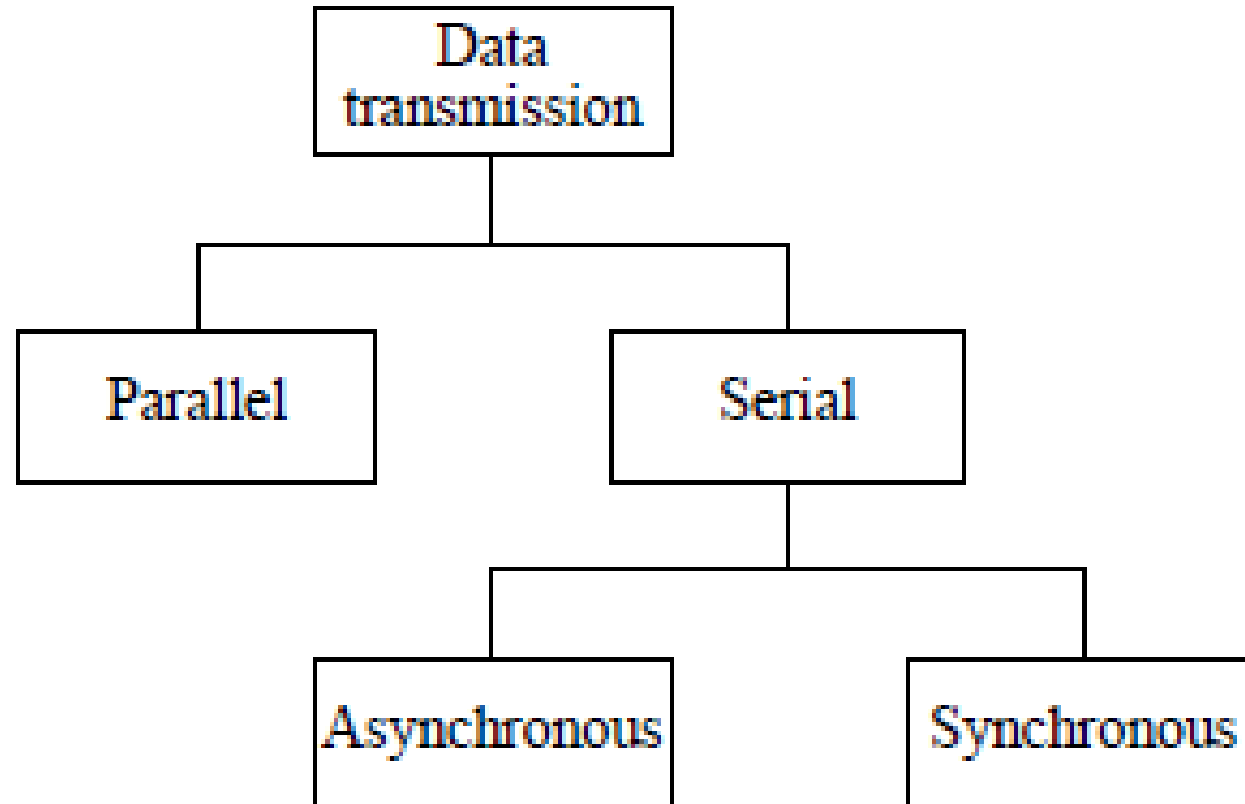


(a) Serial transmission



(b) Parallel transmission

Data Transmission



Serial Communication

- **Synchronous transmission**

- A bus that includes a clock and a fixed protocol for communicating that is relative to the clock
- Disadvantages:
 - All devices on the bus must run at the same clock rate. Done in hardware using phase locked loops (PLLs)
 - Expensive
 - Clock skew problem: cannot be long if they are fast
- Advantages:
 - Block of data can be sent
 - More efficient - Less overhead than asynchronous transmission

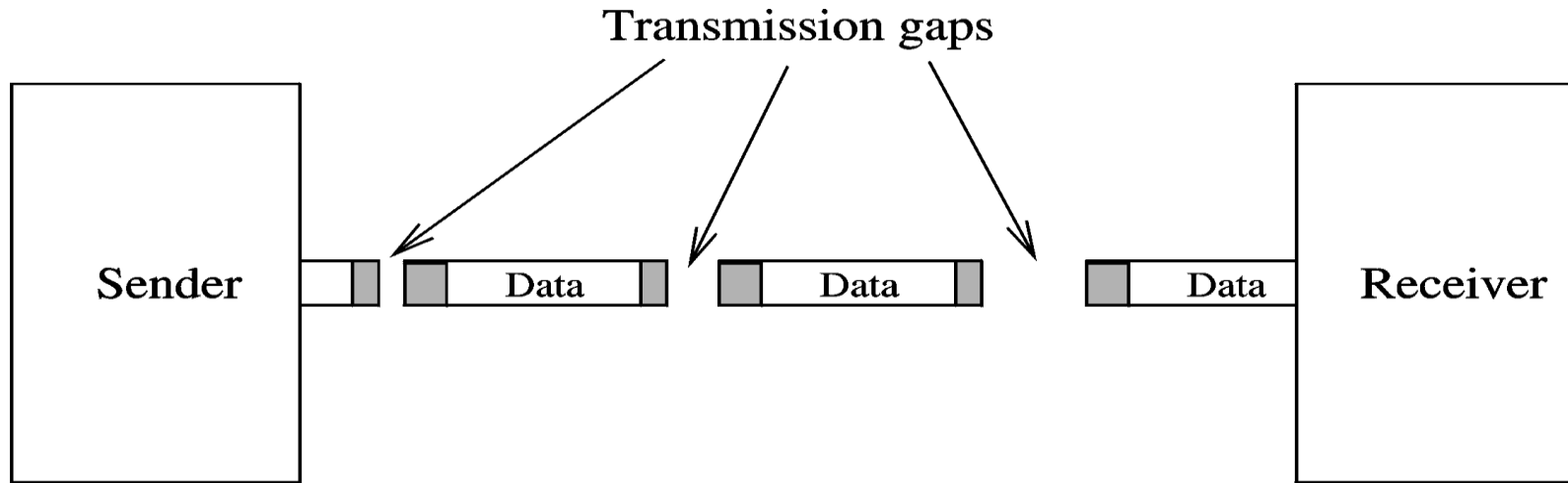
Serial Communication

- **Asynchronous transmission**

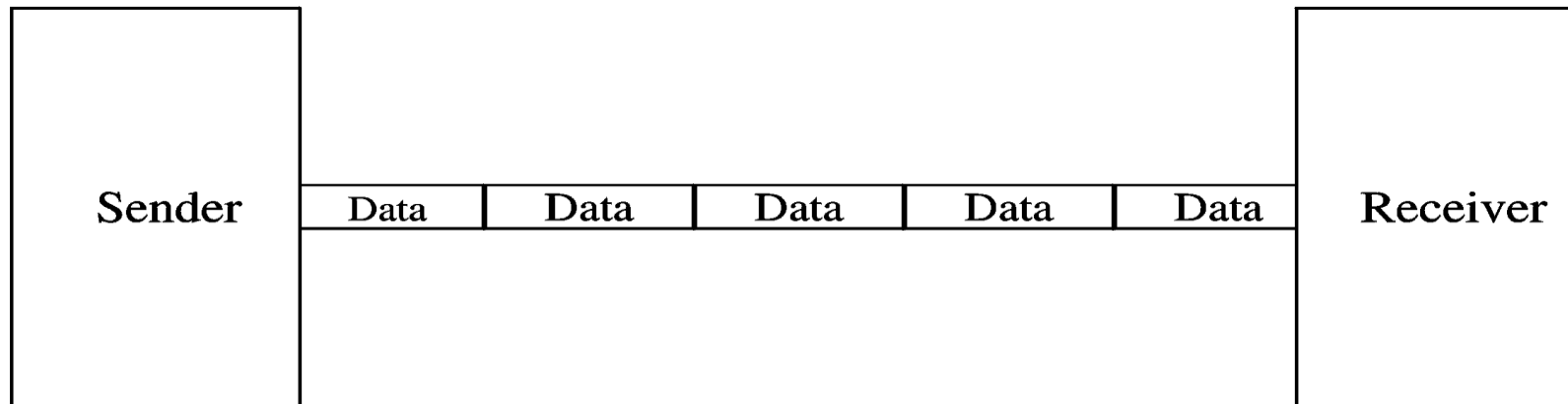
- Each byte of data is enveloped by a **start bit** and one or more **stop bits**
- Uses a **handshaking protocol** for coordinating usage rather than a clock
- Advantages:
 - No need for sender and receiver synchronisation.
 - Can host a wide variety of devices of different speeds.
- Disadvantage:
 - **Slow**: for each eight bits, there are at least two additional bits of overhead, that is, 25% overhead, excluding the gaps between bytes.

- **Low-speed serial transmission** is suitable for I/O devices like the keyboard. Since the characters are typed at a slow speed with large variable gaps, synchronous transmission is not useful.
- **High-speed serial transmission** buses (Universal Serial Bus (USB) and Firewire) use the **synchronous** transmission.

Serial Communication



(a) Asynchronous transmission



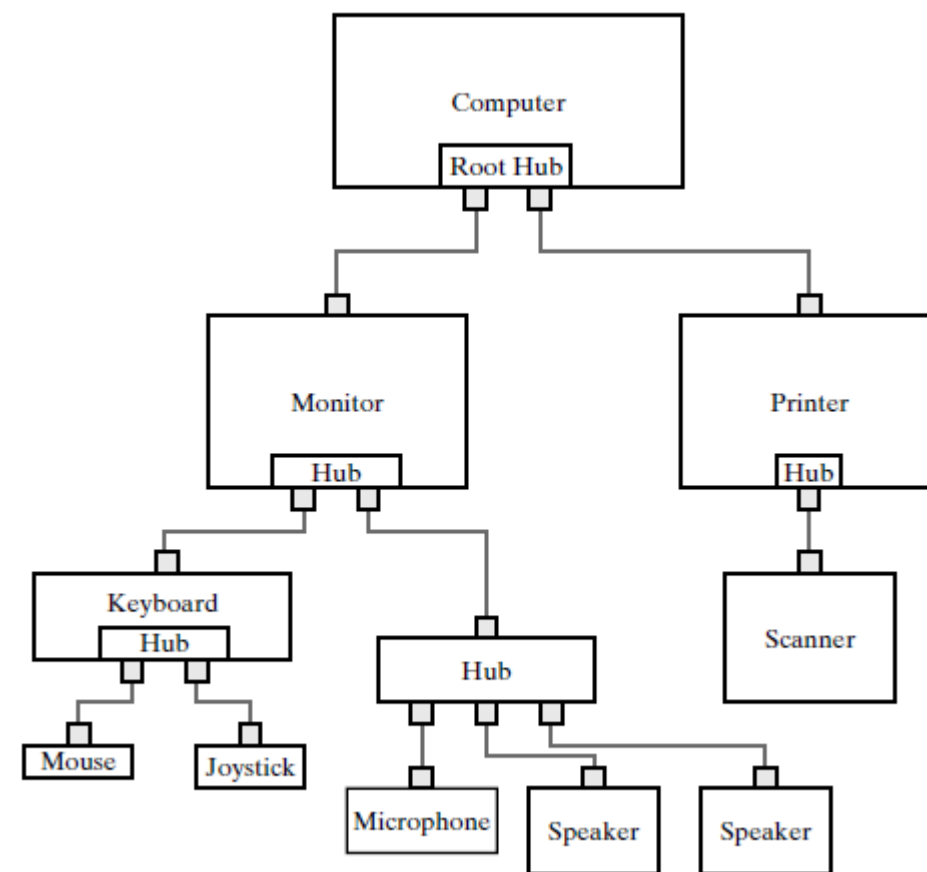
(b) Synchronous transmission

Universal Serial Bus (USB)

- Intended to **help retire** all legacy varieties of **serial and parallel ports**.
 - **Single standardized interface socket**
 - **Plug-and-play + hot swapping**
 - **Providing power** to low-consumption devices
 - Allowing many **standard devices to be used** without requiring manufacturer specific
- Additional advantages of USB
 - **Power distribution**
 - Simple devices can be bus-powered (Examples: mouse, keyboards, wireless LANs)
 - **Control peripherals**
 - Possible because USB allows data to flow in both directions
 - **Expandable through hubs**
 - **Power conservation**
 - Enters suspend state if there is no activity for 3 ms.
 - **Error detection and recovery**

Universal Serial Bus (USB)

- The computer and attached devices can be classified as **hubs, devices, or compound devices**.
- A **hub** provides attachment points for USB devices and other hubs. It contains a USB interface for control and status handling.
- If a hub is combined with a device such as the keyboard in the figure, the keyboard is referred to as a **compound device**.

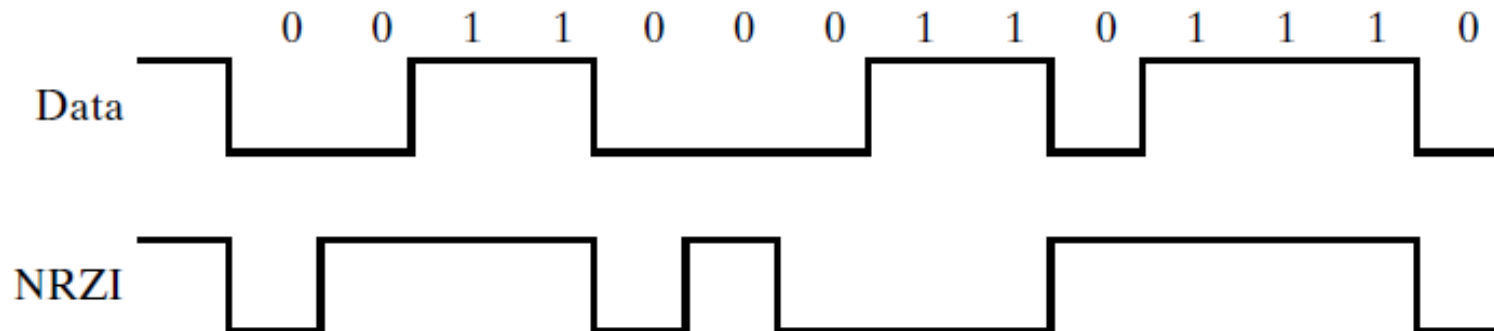


Universal Serial Bus (USB)

- The USB cables contains **four wires**: ground, power, and two data lines (D+ and D–) used for differential signaling.
- The **power wire** is used to **provide small amounts of power** to devices such as keyboards so that they do not need their own power supplies.
- The use of **twisted pairs and differential signaling** reduces the effects of external interference.
- To provide immunity to signal variation and noise, **0s and 1s** are transmitted by using the difference in voltage between **D+ and D–**.
 - If the voltage on **D+ exceeds the voltage on D– by 200 millivolts** or more, then the logic value is a **1**.
 - If the voltage on **D– exceeds the voltage on D+ by 200 millivolts** or more, the logic value is a **0**.
 - Other voltage relationships between D+ and D– are used as special signal states as well.
- *Note: For USB 3, new lines were introduced. (Transmitted data: TX1+ & TX1- and received data: RX1+ & RX1- and RX2+ & RX2-*

Universal Serial Bus (USB)

- The logic values used for signaling are not the actual logic values of the information being transmitted. Instead, a **Non-Return-to-Zero Inverted (NRZI)** signaling convention is used.
- A **zero** in the data being transmitted is represented by a **transition** from 1 to 0 or 0 to 1.
- A **1** is represented by a **fixed value** of 1 or 0.

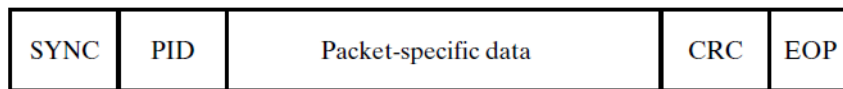


Universal Serial Bus (USB)

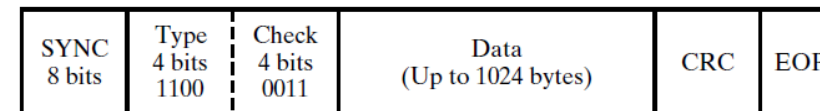
- As is typical for I/O devices, there is **no common clock** serving both the computer and the device.
- **NRZI encoding** of the data provides edges that can be used to maintain **synchronization** between the arriving data and the time at which each bit is sampled at the receiver.
- If there are a **large number of 1s** in series in the data, there will be **no transitions for some time** in the NRZI encoding.
- To **prevent loss of synchronization**, a **0** is “stuffed” in before every seventh bit position in a string of 1s prior to NRZI encoding so that no more than six 1s appear in series. The receiver must be able to remove these extra zeros when converting NRZI data to normal data.

USB Transmission

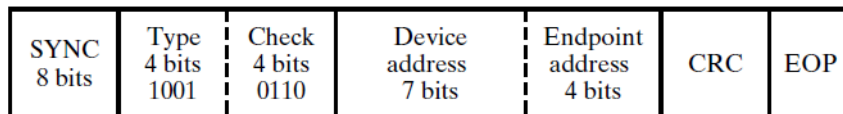
- USB information is transmitted in packets. Each packet contains a specific set of fields, depending on the packet type.
- For example, an output transaction consists of an Out packet followed by a Data packet and a Handshake packet.
- The **Out packet** comes from the USB controller in the computer and notifies the device that it is to receive data.
- The computer then sends the **Data packet**.
- If the Data packet is received without error, then the device responds with the **Acknowledge Handshake packet**.



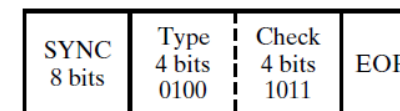
(a) General packet format



(c) Data packet (Data0 type)



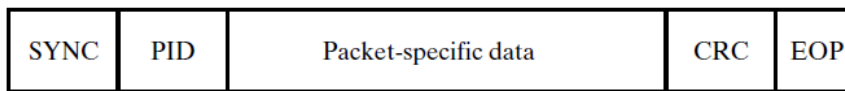
(b) Output packet



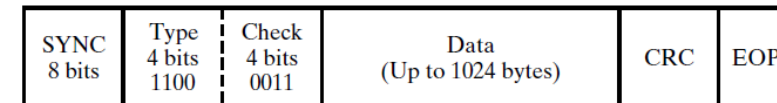
(d) Handshake packet (Acknowledge type)

USB Transmission

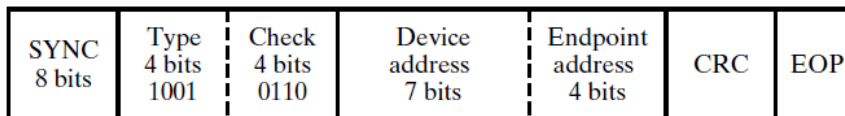
- Note that each packet begins with a synchronization pattern **SYNC**. This pattern is 00000001. Because of the sequence of zeros, the corresponding NRZI pattern contains seven edges, which provides a pattern to which the receiving clock can be synchronized. The pattern also signals the beginning of a new packet.
- Following the SYNC, each packet format contains eight bits called the packet identifier (PID). In the **PID**, the packet type is specified by four bits, with an additional four bits that are complements of the first four to provide an error check on the type.
- As there are several different types of data that can be sent, a token indicating the **type** is required.



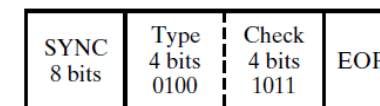
(a) General packet format



(c) Data packet (Data0 type)



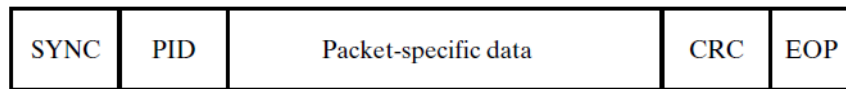
(b) Output packet



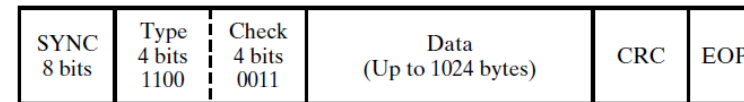
(d) Handshake packet (Acknowledge type)

USB Transmission

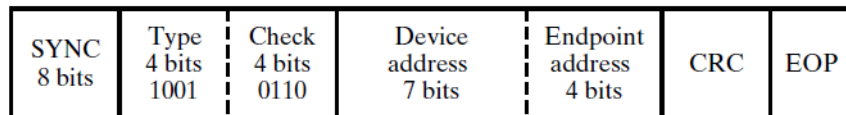
- Optionally, a **CRC** field appears next. The CRC pattern consisting of five or 16 bits is a **Cyclic Redundancy Check** pattern. This pattern is calculated at transmission of the packet from the packet-specific data. The same calculation is performed when the data is received. If the CRC pattern does not match the newly calculated pattern, then an error has been detected. In response to the error, the packet can be ignored and retransmitted.
- In the last field of the packet, an **End of Packet (EOP)** appears. As its name indicates, this sequence of signal states identifies the end of the current packet.



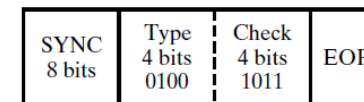
(a) General packet format



(c) Data packet (Data0 type)



(b) Output packet



(d) Handshake packet (Acknowledge type)

Universal Serial Bus (USB)

- The USB supports the following four transfer types:
 - interrupt, isochronous, control, and bulk.
- **Interrupt Transfer**
 - The USB does not support interrupts in the traditional sense.
 - For example, the keyboard generates an interrupt whenever you press or release a key. Instead, **the USB uses polling**, which is similar to the busy-waiting used by the programmed I/O. To get acceptable performance, we have to select an appropriate polling frequency.
- **Isochronous Transfer**
 - Use by **real-time applications** that require a constant data transfer rate.
 - These applications need data in a timely manner. For example, live audio channels.
 - It **does not use and data checking**, as there is not time to resend any data packets with errors

Universal Serial Bus (USB)

- **Control Transfer**

- Control transfers are used to **configure and set up** the USB devices.
- A control transfer involves two to three steps:
 - **Setup stage**: to **convey the type of request made** to the target device (e.g., reading the contents of the device descriptor).
 - **Data Stage**: This is optional and only control transfers that **require data transfers** use this stage. For example, when the request is to read the device descriptor, the device will use this stage to send the descriptor.
 - **Status Stage**: used to **check the status** of the operation.

- **Bulk Transfer**

- Devices that **do not have any specific transfer rate** requirements use the bulk transfers. For example, in transferring a file to a printer, the data can be transferred at a slower rate without any problem.
- The data integrity is verified using cyclic redundancy checking, CRC and an acknowledgement is sent.

Interrupts and Exceptions

- **Interrupts** are external events that require the processor's attention
 - *Examples:* I/O device needs attention, timer interrupts to mark cycle
- The mechanism of interrupt is similar to **procedure call**, but interrupts can be initiated by both software and hardware.
 - Procedures are purely software-initiated.
- Interrupts are normal, not errors
 - All interrupts are recoverable: interrupted program will need to be resumed after the interrupt is handled.
- The MIPS processor uses the system call (`syscall`) instruction for software interrupts.
 - We used `syscall` to invoke the SPIM simulator I/O services.
- The interrupt service routine is commonly called *interrupt handler*.

Interrupts and Exceptions

- **Exceptions** are typically errors that are detected within the processor
 - *Examples:* illegal instruction opcode, arithmetic overflow, or attempts to divide by 0.
- There are two possible ways of resolving these errors:
 - If the error is **un-recoverable**, the operating system kills the program
 - Less serious problems can often be fixed by OS or the program itself
- The underlying mechanism of interrupts and exceptions is essentially the same.
- OS responsible to do the right thing, such as:
 - Save the current state and shut down the hardware devices
 - Find and load the correct data from the hard disk
 - Transfer data to/from the I/O device, or install drivers

Interrupt Processing in MIPS

- On interrupts
 - Disables further interrupts
 - Record the cause of interrupts/exception – Cause register
 - Transfers control to exception handler located at a 0x80000180 (fixed address)
 - Handler saves the context
 - Program counter (PC)
 - Current operating mode (user or supervisor)
 - Status of interrupts (enables or disabled)
 - Stores return address
 - Uses a register: EPC (exception program counter) register

Exception	Cause
Hardware Interrupt	0x00000000
System Call	0x00000020
Breakpoint / Divide by 0	0x00000024
Undefined Instruction	0x00000028
Arithmetic Overflow	0x00000030

MIPS exception cause codes

Interrupts Processing in MIPS

- **Cause** and **EPC** registers are loaded into **k0** and **k1** registers

```
mfc0    $k0,$13    # copy Cause register (Register 13) into $k0  
mfc0    $k1,$14    # copy EPC register (Register 14) into $k1
```

- Cause and EPC registers are not part of the MIPS register file.
 - Located in coprocessor0 (CP0)
- The processor returns to the address in EPC after handling the exception.
 - Return address = address of the interrupted instruction
 - We need to add 4 to get to the instruction following the interrupted instruction

```
addiu    $k1,$k1,4  
rfe  
jr       $k1
```



Q & A
Thank you